

Министерство науки и высшего образования Российской Федерации
ИТМО

Лабораторная работа №5

C++ OOP / PARALLEL

по дисциплине:
"C++ и UNIX системы"

Выполнил:
Студент 3 курса, группы Z3300
Заварина Софья

Преподаватель:
Маслов И.Д.

Санкт-Петербург
2026

Цель

Познакомить студента с принципами объектно-ориентированного программирования на примере создания сложной синтаксической структуры. Придумать синтаксис своего персонального миниязыка параллельного программирования, а также реализовать его разбор и вычисление.

Задача и решение

1. [C++ PARALLEL LANG] Создать параллельный язык программирования

Требуется создать язык программирования, в котором будет доступна установка следующих команд:

- Установка счетного цикла
- Вывод в консоль
- Вывод в файл в режиме добавления
- Арифметические операции $+$, $-$, $*$, $/$

Счетный цикл должен поддерживать дальнейшую установку всех остальных поддерживаемых команд.

Для реализации задачи использовать технологию объектно-ориентированного программирования в части реализации поддерживаемых команд языка.

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <memory>
#include <string>
#include <cctype>
#include <unordered_map>
#include <stdexcept>
#include <thread>
#include <mutex>
#include <chrono>
#include <ctime>
#include <iomanip>
#include <windows.h>

//Вспомогательные утилиты

// Удаление пробелов в начале строки
static void trimLeft(const char* p) {
    while (*p && std::isspace(static_cast<unsigned char>(*p))) ++p;
}

// Контекст выполнения
struct Context {
    std::unordered_map<std::string, int> vars;
    std::mutex& file_mutex;
    int line_number; // для идентификации в логах
    Context(std::mutex& fm, int ln) : file_mutex(fm), line_number(ln) {}
};
```

```

// Иерархия выражений
struct Expr {
    virtual ~Expr() = default;
    virtual int eval(const Context& ctx) const = 0;
};

struct NumExpr : Expr {
    int val;
    explicit NumExpr(int v) : val(v) {}
    int eval(const Context&) const override { return val; }
};

struct VarExpr : Expr {
    std::string name;
    explicit VarExpr(std::string n) : name(std::move(n)) {}
    int eval(const Context& ctx) const override {
        auto it = ctx.vars.find(name);
        if (it == ctx.vars.end())
            throw std::runtime_error("Unknown variable: " + name);
        return it->second;
    }
};

struct BinOpExpr : Expr {
    enum Op { ADD, SUB, MUL, DIV } op;
    std::unique_ptr<Expr> left, right;
    BinOpExpr(Op o, std::unique_ptr<Expr> l, std::unique_ptr<Expr> r)
        : op(o), left(std::move(l)), right(std::move(r)) {}
    int eval(const Context& ctx) const override {
        int l = left->eval(ctx);
        int r = right->eval(ctx);
        switch (op) {
            case ADD: return l + r;
            case SUB: return l - r;
            case MUL: return l * r;
            case DIV:
                if (r == 0) throw std::runtime_error("Division by zero");
                return l / r;
        }
        return 0;
    }
};

// Иерархия команд
struct Command {
    virtual ~Command() = default;
    virtual void execute(Context& ctx) const = 0;
};

using CommandList = std::vector<std::unique_ptr<Command>>;

// Печать в консоль
struct PrintCommand : Command {
    std::unique_ptr<Expr> expr;
    explicit PrintCommand(std::unique_ptr<Expr> e) : expr(std::move(e)) {}
};

```

```

    void execute(Context& ctx) const override;
};

// Печать в файл
struct PrintFileCommand : Command {
    std::string filename;
    std::unique_ptr<Expr> expr;
    PrintFileCommand(std::string fname, std::unique_ptr<Expr> e)
        : filename(std::move(fname)), expr(std::move(e)) {}
    void execute(Context& ctx) const override;
};

// Цикл
struct LoopCommand : Command {
    std::string var;
    std::unique_ptr<Expr> start, end;
    CommandList body;
    LoopCommand(std::string v, std::unique_ptr<Expr> s, std::unique_ptr<Expr> e, CommandList b)
        : var(std::move(v)), start(std::move(s)), end(std::move(e)), body(std::move(b)) {}
    void execute(Context& ctx) const override;
};

// Глобальные мьютексы для синхронизации вывода
std::mutex cout_mutex;
std::mutex file_mutex;

// Вывод времени с префиксом потока
static void log_time(const std::string& prefix, int line_no) {
    auto now = std::chrono::system_clock::now();
    auto t = std::chrono::system_clock::to_time_t(now);
    auto ms = std::chrono::duration_cast<std::chrono::milliseconds>(
        now.time_since_epoch()) % 1000;
    std::lock_guard<std::mutex> lock(cout_mutex);
    std::cout << "[Thread for line " << line_no << "] " << prefix << " at "
        << std::put_time(std::localtime(&t), "%T") << ' '
        << std::setfill('0') << std::setw(3) << ms.count() << '\n';
}

// Реализация команд
void PrintCommand::execute(Context& ctx) const {
    int val = expr->eval(ctx);
    std::lock_guard<std::mutex> lock(cout_mutex);
    std::cout << "[Line " << ctx.line_number << "] print: " << val << std::endl;
}

void PrintFileCommand::execute(Context& ctx) const {
    int val = expr->eval(ctx);
    std::lock_guard<std::mutex> lock(ctx.file_mutex);
    std::ofstream f(filename, std::ios::app);
    if (!f) throw std::runtime_error("Cannot open file: " + filename);
    f << val << std::endl;
}

void LoopCommand::execute(Context& ctx) const {
    int s = start->eval(ctx);
    int e = end->eval(ctx);

```

```

    for (int i = s; i <= e; ++i) {
        // Сохраняем предыдущее значение переменной, если было
        auto old = ctx.vars.find(var);
        int old_val = 0;
        bool had = (old != ctx.vars.end());
        if (had) old_val = old->second;
        ctx.vars[var] = i;    // устанавливаем локальную переменную цикла
        try {
            for (auto& cmd : body)
                cmd->execute(ctx);
        } catch (...) {
            // Восстанавливаем переменную и пробрасываем исключение
            if (had) ctx.vars[var] = old_val; else ctx.vars.erase(var);
            throw;
        }
        if (had) ctx.vars[var] = old_val; else ctx.vars.erase(var);
    }
}

// Парсер
class Parser {
public:
    explicit Parser(const std::string& input) : p_(input.c_str()) {}

    // программа = оператор ';' оператор*
    CommandList parseProgram() {
        CommandList prog;
        while (true) {
            trimLeft(p_);
            if (*p_ == '\0' || *p_ == '}') break;
            prog.push_back(parseStatement());
            trimLeft(p_);
            if (*p_ == ';') { ++p_; continue; }
            if (*p_ != '\0' && *p_ != '}')
                throw std::runtime_error("Expected ';' or end of block");
        }
        return prog;
    }

private:
    const char* p_;

    // оператор = "print" выражение | "printfile" строка выражение | "loop" ...
    std::unique_ptr<Command> parseStatement() {
        trimLeft(p_);
        if (std::string(p_, 5) == "print" && std::isspace(p_[5])) {
            p_ += 5;
            auto expr = parseExpr();
            return std::make_unique<PrintCommand>(std::move(expr));
        }
        if (std::string(p_, 9) == "printfile" && std::isspace(p_[9])) {
            p_ += 9;
            trimLeft(p_);
            // читаем имя файла (до пробела)
            std::string fname;
            while (*p_ && !std::isspace(static_cast<unsigned char>(*p_))) {

```

```

        fname += *p_;
        ++p_;
    }
    if (fname.empty()) throw std::runtime_error("Expected filename after printfile");
    auto expr = parseExpr();
    return std::make_unique<PrintFileCommand>(fname, std::move(expr));
}

if (std::string(p_, 4) == "loop" && std::isspace(p_[4])) {
    p_ += 4;
    trimLeft(p_);
    // идентификатор
    std::string var;
    while (*p_ && (std::isalnum(static_cast<unsigned char>(*p_)) || *p_ == '_')) {
        var += *p_;
        ++p_;
    }
    if (var.empty()) throw std::runtime_error("Expected loop variable name");
    trimLeft(p_);
    if (*p_ != '=') throw std::runtime_error("Expected '=' after loop variable name");
    ++p_;
    auto startExpr = parseExpr();
    trimLeft(p_);
    if (*p_ != ',') throw std::runtime_error("Expected ',' after loop start value");
    ++p_;
    auto endExpr = parseExpr();
    trimLeft(p_);
    if (*p_ != '{') throw std::runtime_error("Expected '{' before loop body");
    ++p_;
    CommandList body = parseProgram(); // парсим тело до '}'
    trimLeft(p_);
    if (*p_ != '}') throw std::runtime_error("Expected '}' at end of loop body");
    ++p_;
    return std::make_unique<LoopCommand>(var, std::move(startExpr), std::move(endExpr), std::move(body));
}

throw std::runtime_error(std::string("Unknown command starting with: ") + p_);
}

// выражение = слагаемое ( '+' '-' ) слагаемое *
std::unique_ptr<Expr> parseExpr() {
    auto left = parseTerm();
    while (true) {
        trimLeft(p_);
        if (*p_ == '+') { ++p_; left = std::make_unique<BinOpExpr>(BinOpExpr::ADD, std::move(left), parseTerm()); }
        else if (*p_ == '-') { ++p_; left = std::make_unique<BinOpExpr>(BinOpExpr::SUB, std::move(left), parseTerm()); }
        else break;
    }
    return left;
}

// слагаемое = множитель ( '*' '/' ) множитель *
std::unique_ptr<Expr> parseTerm() {
    auto left = parseFactor();
    while (true) {
        trimLeft(p_);
        if (*p_ == '*') { ++p_; left = std::make_unique<BinOpExpr>(BinOpExpr::MUL, std::move(left), parseFactor()); }
        else if (*p_ == '/') { ++p_; left = std::make_unique<BinOpExpr>(BinOpExpr::DIV, std::move(left), parseFactor()); }
        else break;
    }
    return left;
}

```

```

        else break;
    }
    return left;
}

// множитель = число | идентификатор | '(' выражение ')',
std::unique_ptr<Expr> parseFactor() {
    trimLeft(p_);
    if (*p_ == '(') {
        ++p_;
        auto e = parseExpr();
        trimLeft(p_);
        if (*p_ != ')') throw std::runtime_error("Expected ')')");
        ++p_;
        return e;
    }
    if (std::isdigit(static_cast<unsigned char>(*p_))) {
        int val = 0;
        while (std::isdigit(static_cast<unsigned char>(*p_))) {
            val = val * 10 + (*p_ - '0');
            ++p_;
        }
        return std::make_unique<NumExpr>(val);
    }
    if (std::isalpha(static_cast<unsigned char>(*p_)) || *p_ == '_') {
        std::string name;
        while (*p_ && (std::isalnum(static_cast<unsigned char>(*p_)) || *p_ == '_')) {
            name += *p_;
            ++p_;
        }
        return std::make_unique<VarExpr>(name);
    }
    throw std::runtime_error(std::string("Unexpected character in expression: ") + *p_);
}

};

//Функция выполнения одного набора команд (в отдельном потоке) =====
void execute_line(int line_no, const std::string& line) {
    log_time("Start", line_no);
    try {
        Parser parser(line);
        CommandList program = parser.parseProgram();
        Context ctx(file_mutex, line_no);
        for (auto& cmd : program)
            cmd->execute(ctx);
    } catch (const std::exception& e) {
        std::lock_guard<std::mutex> lock(cout_mutex);
        std::cerr << "[Line " << line_no << "] Error: " << e.what() << std::endl;
    }
    log_time("Finish", line_no);
}

// Главная функция
int main() {
    SetConsoleOutputCP(65001); // Установить UTF-8 для вывода
    SetConsoleCP(65001);      // Установить UTF-8 для ввода

```

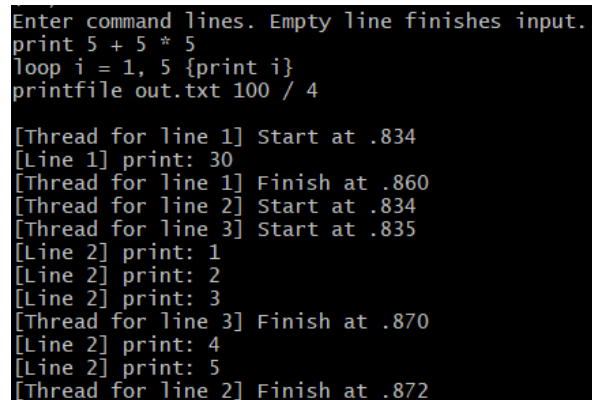
```

std::vector<std::string> lines;
std::cout << "Enter command lines. Empty line finishes input.\n";
std::string input;
while (std::getline(std::cin, input)) {
    if (input.empty()) break;
    // Удаляем возможные символы возврата каретки и т.п.
    while (!input.empty() && (input.back() == '\r' || input.back() == '\n'))
        input.pop_back();
    if (input.empty()) break;
    lines.push_back(input);
}
if (lines.empty()) {
    std::cout << "No commands to execute.\n";
    return 0;
}

std::vector<std::thread> threads;
for (size_t i = 0; i < lines.size(); ++i) {
    threads.emplace_back(execute_line, static_cast<int>(i + 1), lines[i]);
}
for (auto& th : threads) th.join();

return 0;
}

```



```

Enter command lines. Empty line finishes input.
print 5 + 5 * 5
loop i = 1, 5 {print i}
printf out.txt 100 / 4

[Thread for line 1] Start at .834
[Line 1] print: 30
[Thread for line 1] Finish at .860
[Thread for line 2] Start at .834
[Thread for line 3] Start at .835
[Line 2] print: 1
[Line 2] print: 2
[Line 2] print: 3
[Thread for line 3] Finish at .870
[Line 2] print: 4
[Line 2] print: 5
[Thread for line 2] Finish at .872

```

Рис. 1: Вывод

[LOG] Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

Заключение

В ходе лабораторной работы я создала свой параллельный язык программирования на C++ — с помощью объектно-ориентированного подхода.

В целом работа дала понимание, как устроены языки программирования изнутри — от текста до выполнения.